



KANDÓ KÁLMÁN INFORMATIKAI TECHNIKUM

Alkotások Tára – Könyvtárkezelő rendszer

Készítette

Tóth Kende Balázs

Prion Zalán

Somogyi Kristóf

Szoftverfejlesztő- és tesztelő szak

Témavezető

Négyesi Gábor

konzulens

MISKOLC, 2026

Tartalomjegyzék

1. Fejlesztői környezet	5
1.1. Frontend fejlesztői környezete	5
1.1.1. React 19	5
1.1.2. JavaScript (ES6+)	5
1.1.3. HTML5 és CSS3	5
1.1.4. Bootstrap 5	6
1.1.5. React Router v7	6
1.1.6. React Context API	6
1.2. Backend fejlesztői környezete	6
1.2.1. C#	6
1.2.2. .NET 8	7
1.2.3. ASP.NET Core	7
1.2.4. MySQL (MariaDB)	7
1.2.5. Entity Framework Core 8	7
1.2.6. REST API	7
1.2.7. AutoMapper 12.0.1	7
1.2.8. FluentValidation 11.3.1	8
1.2.9. MailKit 4.14.1	8
1.2.10. Serilog 9.0.0	8
1.2.11. BCrypt.Net 4.0.3	8
1.3. Desktop alkalmazás fejlesztői környezete	8
1.3.1. .NET 10.0	8
1.3.2. WPF + Material Design 3	8
1.3.3. MVVM architektúra	8
1.3.4. LiveChartsCore 2.0.0	9
1.3.5. Dependency Injection	9
2. Csapatmunka és Projektmenedzsment	10
2.1. Git – Verziókezelés és kollaboráció	10
2.2. Trello – Feladatkezelés és projektstruktúra	10
2.3. Discord – Kommunikáció és gyors egyeztetések	10
2.4. Személyes együttműködés – Tanórai és órán kívüli kooperáció	11

3. Felhasználói dokumentáció	12
3.1. Motiváció és tervezés	12
3.2. Célközönség	12
3.3. Hasonló rendszerek	12
3.4. Innováció és alkalmazhatóság	13
3.5. Fejlesztési feladatmegosztás	13
4. Projektről	14
5. Fejlesztési dokumentáció	15
5.1. Frontend	15
5.1.1. Bevezető	15
5.1.2. Első lépések a frontend fejlesztésében	15
5.1.3. Főoldal (HomePage)	16
5.1.4. Bejelentkezés és regisztráció (LoginPage)	17
5.1.5. Könyvkatalógus (BooksPage)	19
5.1.6. Kosár és fizetési rendszer (CartPage)	19
5.1.7. Profiloldal (ProfilePage)	20
5.1.8. Adminisztrátori felület (AdminPage)	20
5.1.9. WPF karbantartó alkalmazás	21
5.1.10. Bérleti rendszer és értesítések	23
5.1.11. Példánykészlet-kezelés	24
5.1.12. Tesztelés és minőségbiztosítás	24
5.2. Adatbázis	24
5.2.1. Bevezető	24
5.2.2. Táblák részletes leírása	25
5.2.3. Táblakapcsolatok összefoglalása	28
5.3. Backend	30
5.3.1. Bevezető	30
5.3.2. Hitelesítés és jogosultságkezelés (JWT)	30
5.3.3. Könyvkezelés	33
5.3.4. Szerzők kezelése	33
5.3.5. Példánykezelés	33
5.3.6. Kosár és fizetés	33
5.3.7. Kölcsönzések kezelése	33
5.3.8. Felhasználók kezelése	33
6. REST API végpontok összefoglalása	34
6.1. AuthController (/api/auth)	34
6.2. BooksController (/api/books)	34

6.3.	AuthorController (/api/author)	35
6.4.	CopiesController (/api/copies)	36
6.5.	CartController (/api/cart)	36
6.6.	RentalsController (/api/rentals)	37
6.7.	PaymentsController (/api/payments)	37
6.8.	UsersController (/api/users)	38
7.	Projektstruktúra	39
	Irodalomjegyzék	40

1. fejezet

Fejlesztői környezet

1.1. Frontend fejlesztői környezete

1.1.1. React 19

A React egy nyílt forráskódú JavaScript könyvtár, amelyet a Facebook fejlesztett az interaktív felhasználói felületek hatékony megépítésére. Az Alkotások Tára weboldal teljes frontendje React 19.2.0 komponensekből áll, amelyek deklaratív, komponens-alapú megközelítést követnek. A React lehetővé teszi az alkalmazás állapotának kezelését (State Management) az AuthContext, CartContext és ToastContext segítségével, az újrafelhasználható komponensek összeállítását, valamint az egyoldalas alkalmazás (SPA) architektúrájának megvalósítását. A projekt JavaScript nyelven íródott (nem TypeScript), és a Create React App (CRA) eszközzel lett inicializálva, amely biztosítja az egyszerű fejlesztési és build folyamatot. Ez a technológia garantálja, hogy a weboldal gyors, reszponzív és könnyen karbantartható marad.

1.1.2. JavaScript (ES6+)

A frontend JavaScript ES6+ szintaxissal készült, amely modern, olvasható és hatékony kódot tesz lehetővé. Az arrow function-ök, const/let deklarációk, destructuring, template literals és async/await aszinkron programozás az alapvető eszközöknek részei. Ez a nyelvváltozat biztosítja, hogy a fejlesztési folyamat produktív, gyors és biztonságos legyen, valamint a kódminőség magas szinten maradjon.

1.1.3. HTML5 és CSS3

A weboldal szerkezetét HTML5 szemantikus elemek adják, amely modern, hozzáférhető és SEO-barát felépítést biztosít. A CSS3 stílusleírás reszponzív designt valósít meg, media query-k segítségével alkalmazkodva asztali számítógépekhez, tabletekhez és mobil eszközökhöz. A frontend dizájnja egy minimalista fekete-fehér témára épül (#1a1a1a

háttér, #fafafa előttér), szögletes gombok stílussal, amely egyedi és karakteres megjelenést biztosít.

1.1.4. Bootstrap 5

A weboldal reszponzív megjelenését a Bootstrap 5.3.8 biztosítja, amely gyors és egységes UI-felépítést tesz lehetővé. A jól strukturált grid rendszer, valamint a kész komponensek (gombok, navigációk, kártyák) segítik a gyors fejlesztést és a letisztult dizájnt. A Bootstrap mobile-first megközelítése garantálja, hogy az oldal minden eszközön azonnal használható és modern megjelenést kapjon. A Bootstrap Icons 1.13.1 ikonkészlet egységes vizuális megjelenést biztosít az alkalmazás egészében.

1.1.5. React Router v7

A kliensoldali navigáció a React Router DOM 7.13.0 verzióval valósul meg. A Browser-Router használata lehetővé teszi a SPA (Single Page Application) jellegű oldalváltásokat teljes oldalújrátöltés nélkül, javítva a felhasználói élményt. A könyvek böngészése, a kosár, a bejelentkezés, az admin felület, a profil és az email verifikáció mind külön útvonalakon érhető el.

1.1.6. React Context API

Az alkalmazás állapotkezelése a React beépített Context API-ján alapul, három fő kontextussal:

- **AuthContext** – felhasználói hitelesítés és munkamenet-kezelés
- **CartContext** – kosár állapot és műveletek kezelése
- **ToastContext** – értesítési üzenetek (toast) megjelenítése

1.2. Backend fejlesztői környezete

1.2.1. C#

A backend fejlesztés C# programozási nyelven történik, amely típusbiztos, objektum-orientált nyelvet biztosít. A C# erős típusellenőrzése és teljesítménye ideális vállalati alkalmazásokhoz. A projekt nullable reference types funkciót is használja a biztonságosabb kódírás érdekében. A C# nyelv szintaxisa jól ismert, egyértelmű, és az ASP.NET Core keretrendszerrel szorosan integrálódik, így a fejlesztési folyamat zökkenőmentes és produktív.

1.2.2. .NET 8

A projekt .NET 8 platformon alapul, amely cross-platform támogatást és modern fejlesztési eszközöket biztosít. A .NET 8 a legújabb LTS (Long Term Support) verzió, amely hosszú távú támogatást és magas teljesítményt garantál. Ez a platform biztosítja, hogy a backend skálázható, biztonságos és hosszú ideig karbantartható marad.

1.2.3. ASP.NET Core

Az ASP.NET Core webkeretrendszer használata a REST API fejlesztéséhez. Az ASP.NET Core magas teljesítményt és skálázhatóságot biztosít. A projekt Swagger/OpenAPI dokumentációt is tartalmaz a Swashbuckle.AspNetCore 9.0.6 csomag segítségével, amely interaktív API dokumentációt biztosít a fejlesztők számára.

1.2.4. MySQL (MariaDB)

Az adatbázis-kezeléshez MySQL relációs adatbázis-rendszert használunk, a Pomelo.EntityFrameworkCore.MySql provider segítségével. Az adatbázis a dongeszh_BookStore sémát használja, utf8mb4 karakterkészlettel és utf8mb4_hungarian_ci rendezéssel a magyar nyelvű tartalom helyes kezeléséhez. A tárhely a s58.tarhely.com szerveren található.

1.2.5. Entity Framework Core 8

Az Entity Framework Core ORM (Object-Relational Mapping) eszköz használata az adatbázis műveletek egyszerűsítéséhez. Az EF Core LINQ lekérdezéseket és automatikus migrációkat biztosít. A projekt Code-First megközelítést alkalmaz, ahol a C# modellek alapján generálódnak az adatbázis táblák. A BookStoreContext kezeli a kapcsolatot az adatbázissal.

1.2.6. REST API

A RESTful API tervezési minták használata az előtér-háttér kommunikációhoz. A REST API szabványos HTTP metódusokat (GET, POST, PUT, PATCH, DELETE) és JSON formátumot használ az adatok átviteléhez. A projekt 8 fő kontrollert tartalmaz: Auth, Books, Author, Copies, Cart, Rentals, Payments és Users.

1.2.7. AutoMapper 12.0.1

Az AutoMapper könyvtár az entitások és DTO-k (Data Transfer Objects) közötti automatikus leképezést biztosítja, csökkentve a boilerplate kódot és javítva a karbantarthatóságot.

1.2.8. FluentValidation 11.3.1

A FluentValidation keretrendszer a bemeneti adatok validálásához használt, amely deklaratív és könnyen olvasható validációs szabályokat biztosít a model osztályokhoz.

1.2.9. MailKit 4.14.1

Az e-mail küldés a MailKit könyvtárral valósul meg, amely SMTP protokollon keresztül kommunikál. A fejlesztési fázisban a Mailtrap sandbox szolgáltatást használjuk a teszteléshez, amely biztonságos környezetet biztosít az e-mail funkciók tesztelésére anélkül, hogy valódi e-maileket küldenénk.

1.2.10. Serilog 9.0.0

A naplózás a Serilog strukturált naplózási keretrendszerrel történik, amely részletes és kereshető naplóbejegyzéseket biztosít a hibakereséshez és a rendszer monitorozásához.

1.2.11. BCrypt.Net 4.0.3

A jelszavak biztonságos tárolása BCrypt hash algoritmussal történik, amely az iparági szabványnak megfelelő, szózott egyirányú hash-t alkalmaz.

1.3. Desktop alkalmazás fejlesztői környezete

1.3.1. .NET 10.0

A desktop alkalmazás a legújabb .NET 10.0 keretrendszerre épül, Windows-specifikus célplatformmal (net10.0-windows).

1.3.2. WPF + Material Design 3

A felhasználói felület WPF (Windows Presentation Foundation) technológiával készült, kiegészítve a MaterialDesignThemes 5.3.0 csomaggal, amely modern Material Design 3 megjelenést biztosít sötét témával és Deep Orange színsémával. A WPF XAML alapú felületet biztosít, amely gazdag felhasználói élményt nyújt Windows környezetben.

1.3.3. MVVM architektúra

Az alkalmazás az MVVM (Model-View-ViewModel) tervezési mintát követi a CommunityToolkit.Mvvm 8.4.0 csomag segítségével, amely ObservableObject és RelayCommand implementációkat biztosít a tiszta kódszervezéshez.

1.3.4. LiveChartsCore 2.0.0

Az adatvizualizáció a LiveCharts2 könyvtárral valósul meg, amely interaktív diagramokat és grafikonokat biztosít a Dashboard statisztikák megjelenítéséhez.

1.3.5. Dependency Injection

A szolgáltatások kezelése a Microsoft.Extensions.DependencyInjection IoC konténerrel történik, amely Singleton és Transient életciklusú regisztrációkat alkalmaz a szolgáltatásokhoz és ViewModel-ekhez.

2. fejezet

Csapatmunka és Projektmenedzsment

2.1. Git – Verziókezelés és kollaboráció

A projekt fejlesztése során Git szolgált a verziókezelés alapjául, amely lehetővé tette a folyamatos, biztonságos és jól követhető közös munkát. A külön ágakon (branch) történő fejlesztés biztosította, hogy a weboldal, a backend és a desktop alkalmazás fejlesztése nem akadályozza egymást, miközben a merge folyamatok garantálták a stabil verziók kialakítását. A Git commit-históriája pontosan dokumentálja a haladást, a refaktorálásokat és az egyes funkciók bővítését, ezzel támogatva az átlátható és professzionális fejlesztési folyamatot.

2.2. Trello – Feladatkezelés és projektstruktúra

A Trello a projekt menedzsmentjének központi eszközeként működött. A fejlesztési folyamat feladatokra bontva, listákba rendezve (To Do, In Progress, Done) volt nyomon követhető, ami egyértelmű struktúrát és folyamatos átláthatóságot biztosított. A Trello lehetővé tette a sprint-alapú tervezést, a feladatok priorizálását és az állapotok nyomon követését. Így a frontend, a backend és a desktop alkalmazás moduljai mind külön, jól követhető workflow-ba rendezve készültek el.

2.3. Discord – Kommunikáció és gyors egyeztetések

A projekt során a Discord biztosította a folyamatos kommunikációt. A gyors üzenetváltások, hívások és képernyőmegosztások segítségével a problémák szinte azonnal megoldást kaptak. A webes, backend-es és desktop-os kérdések külön csatornáknak voltak rendszerezve, ami gördülékennyé tette az információáramlást. A Discord alkalmas volt

ötletelésre, hibajavítás közös átnézésére, és általános kapcsolattartásra is, így kulcsfontosságú eszköz volt a csapatmunkában.

2.4. Személyes együttműködés – Tanórai és órán kívüli kooperáció

A digitális eszközök mellett a személyes kommunikáció is kiemelkedő szerepet játszott. Tanórákon, projektmegbeszéléseken és szabadidőben is rendszeresen egyeztettünk, ami jelentősen gyorsította a közös döntéshozatalt. A közvetlen együttműködés lehetőséget adott azonnali visszajelzésekre, ötletcserékre és a kihívások gyors feltárására. Ez a fajta aktív személyes jelenlét nagymértékben hozzájárult a projekt stabil és folyamatos fejlődéséhez.

3. fejezet

Felhasználói dokumentáció

3.1. Motiváció és tervezés

Az Alkotások Tára projekt célja egy átfogó online könyvkereskedési és bérleti rendszer létrehozása. A motiváció a hagyományos könyvesboltok digitalizálása és a modern felhasználói igények kielégítése volt. A tervezés során figyelembe vettük a felhasználói élményt, a teljesítményt és a skálázhatóságot.

A projekt ötlete abból a felismerésből született, hogy a mai digitális világban szükség van egy olyan online platformra, amely egyszerre kínál könyvvásárlási és könyvkölcsönzési lehetőséget, modern webes technológiákra építve. Az alkalmazás nem csupán egy egyszerű webshop, hanem egy teljes körű könyvtárkezelő rendszer, amely adminisztratori és felhasználói funkciókkal egyaránt rendelkezik.

3.2. Célközönség

A projekt elsődleges célközönsége minden korosztály, aki online szeretne könyveket vásárolni vagy kölcsönözni. A könnyen használható, reszponzív felület és a kategorizált könyvlista megkönnyíti a választást. Az adminisztratori felület és a WPF karbantartó alkalmazás a könyvesbolt üzemeltetőinek nyújt hatékony eszközt a katalógus, a bérlések és a fizetések kezeléséhez.

3.3. Hasonló rendszerek

A projekt olyan ismert online könyvboltokból és könyvtári rendszerekből merített inspirációt, mint az Amazon, a Libri, a Bookline vagy a Moly.hu, amelyek könyvlista, kosár és felhasználói fiók alapú rendszereket biztosítanak. A bérleti funkció tekintetében a hagyományos könyvtári rendszerek szolgáltak mintaként, amelyeket modern webes megoldásokba ültettünk át.

3.4. Innováció és alkalmazhatóság

A projekt innovatív elemei több területen is megjelennek:

- **Teljes körű REST API architektúra** a frontend és backend szétválasztásával, amely lehetővé teszi a rendszer könnyű bővíthetőségét
- **Vásárlási és bérleti rendszer egyben** – a kosárba vásárlásra és kölcsönzésre egyaránt el lehet helyezni könyveket, vegyes rendelés is lehetséges
- **Email alapú regisztrációs verifikáció** biztonságos felhasználókezeléshez, MailKit és Mailtrap integráció
- **BCrypt jelszóhashelés** biztonsági szabványoknak megfelelően
- **Automatikus bérleti értesítő rendszer** háttérszolgáltatásként, amely e-mailben emlékeztet a közelgő és lejárt kölcsönzésekre
- **Kategorizált és szűrhető könyvkatalógus** ár, kategória és szerző szerinti szűréssel
- **WPF alapú adminisztrációs alkalmazás** Material Design 3 megjelenéssel, MVVM architektúrával és interaktív Dashboard statisztikákkal
- **Modern technológiai stack:** React 19 + ASP.NET Core 8 + MySQL + WPF .NET 10

3.5. Fejlesztési feladatmegosztás

A projekt egy háromfős fejlesztőcsapat munkája:

- **Tóth Kende Balázs** – felelős a React alapú frontend weboldal teljes megvalósításáért, a felhasználói felületért, a backend kommunikációért, valamint a WPF alapú desktop karbantartó alkalmazás fejlesztéséért.
- **Prion Zalán** – felelős az ASP.NET Core backend implementációjáért.
- **Somogyi Kristóf** – felelős az adatbázis tervezéséért és megvalósításáért.

A rendszer minden része egymásra épül, így a projekt fejlesztése folyamatos együttműködés eredményeként valósult meg.

4. fejezet

Projektről

Az Alkotások Tára egy saját fejlesztésű, online könyvkereskedési és könyvtárkezelő rendszer, amely modern webes technológiákat ötvöz egy átfogó üzleti logikával. A rendszer három fő komponensből áll: egy React alapú webes felület a felhasználók számára, egy ASP.NET Core alapú REST API backend az üzleti logika és adatkezelés ellátására, valamint egy WPF alapú desktop alkalmazás az adminisztrátori feladatok elvégzéséhez.

A webalkalmazás lehetővé teszi a felhasználók számára a könyvek böngészését, keresését, kategória és ár szerinti szűrését, regisztrációt és bejelentkezést e-mail megerősítéssel, kosárba helyezést és fizetést, valamint könyvek kölcsönzését. Az adminisztrátorok a webes felületen és a desktop alkalmazáson keresztül kezelhetik a könyvkatalógust, a felhasználókat, a bérlesek és a fizetéseket. A rendszer egy MySQL adatbázisra épül, amely 11 fő táblát tartalmaz a könyvek, szerzők, kategóriák, példányok, felhasználók, kosarak, bérlek és fizetések nyilvántartásához.

5. fejezet

Fejlesztési dokumentáció

5.1. Frontend

5.1.1. Bevezető

A projekt frontendje egy modern, React 19 alapú webalkalmazás, amely lehetővé teszi a felhasználók számára a könyvek böngészését, vásárlását és kölcsönzését. A webalkalmazás célja egy átlátható, gyors és felhasználóbarát felület biztosítása, amely stabil kapcsolatban áll a backend API-val, és valós időben jeleníti meg az adatokat. A frontend JavaScript nyelven íródott, és a Create React App eszközzel lett inicializálva.

5.1.2. Első lépések a frontend fejlesztésében

A frontend fejlesztése a weboldal alapstruktúrájának kialakításával kezdődött. A megjelenés kialakításához a Bootstrap 5 keretrendszert, valamint egyedi CSS3 stílusokat alkalmaztunk. A minimalista fekete-fehér téma (#1a1a1a háttér, #fafafa előtér) és a szögletes gombok stílusa egyedi vizuális identitást ad az alkalmazásnak. A React komponens-alapú megközelítése lehetővé tette az újrafelhasználható UI elemek létrehozását és a hatékony állapotkezelést.

Az API kommunikáció egy központi api.js fájlban valósul meg, amely 60+ API függvényt tartalmaz a Fetch API és AbortController használatával. A fejlesztési környezetben a frontend a localhost:3000-en fut, és proxy-n keresztül kommunikál a localhost:5000-en futó backend-del.

A jelenlegi frontend az alábbi főbb oldalakat tartalmazza:

A frontend 21 komponenst tartalmaz, amelyek 5 fő kategóriába sorolhatók:

- **Auth komponensek** (Login, Register, EmailVerification) – hitelesítési funkciók
- **Book komponensek** (BookList, BookCard, BookDetails, CategoryFilter) – könyvek megjelenítése és szűrése

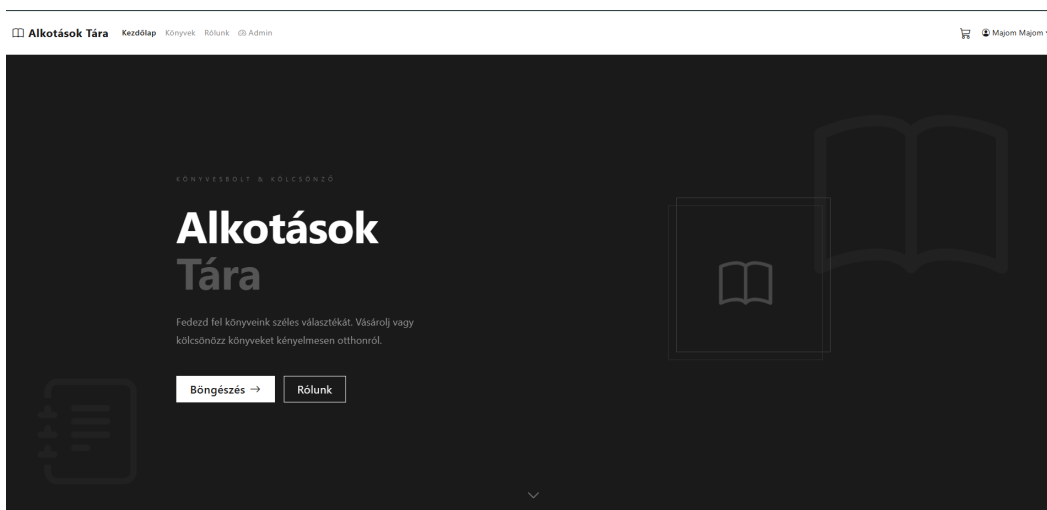
Útvonal	Oldal	Funkció
/	HomePage	Főoldal kiemelt könyvek karusszellel
/books	BooksPage	Könyvkatalógus kereséssel és szűréssel
/about	AboutPage	Információs oldal
/login	LoginPage	Bejelentkezés és regisztráció
/cart	CartPage	Bevásárlókosár
/admin	AdminPage	Adminisztrátori felület
/profile	ProfilePage	Felhasználói profil
/verify	EmailVerification	E-mail megerősítés

5.1. táblázat. Frontend útvonalak

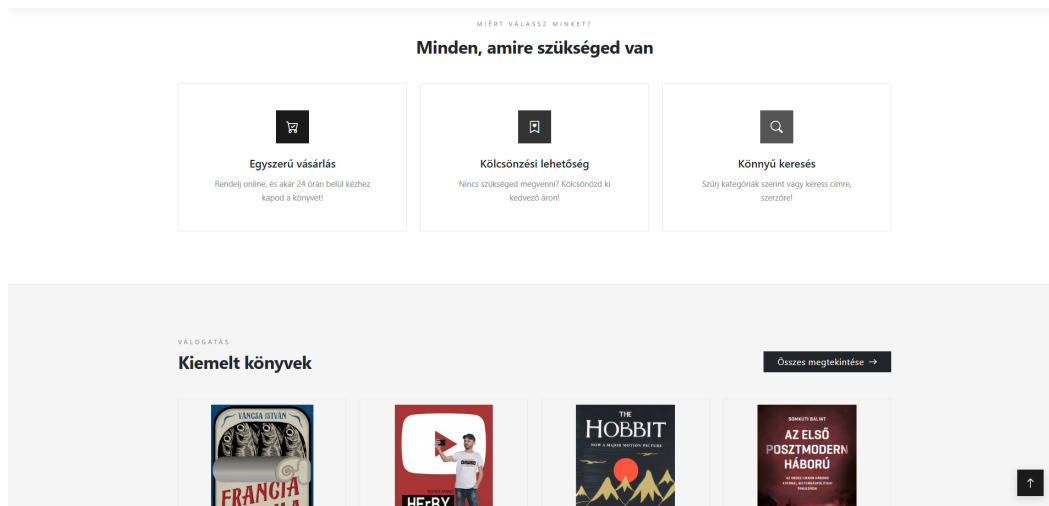
- **Cart komponensek** (Cart, CartItem, Checkout) – kosár és fizetés
- **Admin komponensek** (Dashboard, Statistics, BookManagement, RentalManagement, PurchaseManagement) – adminisztrációs funkciók
- **Common komponensek** (Navbar, Footer, SearchBar, ToastContainer, LoadingSpinner) – közös UI elemek

5.1.3. Főoldal (HomePage)

A főoldal az alkalmazás belépési pontja, amely egy karusszellben jeleníti meg a kiemelt könyveket. A felhasználó innen gyorsan eljuthat a könyvkatalógusba, a bejelentkezési oldalra vagy az információs oldalra. A dizájn a minimalista fekete-fehér témát követi, Bootstrap 5 alapú elrendezéssel.



5.1. ábra. Főoldal – felső rész



5.2. ábra. Főoldal – alsó rész

5.1.4. Bejelentkezés és regisztráció (LoginPage)

Ez az oldal biztosítja a felhasználók számára a regisztráció és a bejelentkezés lehetőséget.

Regisztráció:

- A felhasználó megadja a vezetéknévét, keresztnévét, e-mail címét, jelszavát és címét
- A jelszó BCrypt algoritmussal kerül hash-elésre a szerveren
- A rendszer automatikusan küld egy e-mail megerősítő linket a megadott címre
- A megerősítési token 24 órán belül érvényes

Bejelentkezés:

- E-mail cím vagy felhasználónév + jelszó kombináció
- A felhasználói adatok a böngésző localStorage-ában tárolódnak
- Az oldal újratöltésekor a munkamenet automatikusan helyreáll
- Ha az e-mail még nincs megerősítve, lehetőség van az ellenőrző levél újraküldésére

Jogosultságkezelés:

- Adminisztrátori hozzáférés szerep (role) alapján ellenőrzött
- Az admin felület csak jogosult felhasználók számára érhető el

FIÓK

Bejelentkezés

Email cím

Jelszó

Bejelentkezés

Nincs még fiókod? [Regisztráció](#)

Tipp: Használj email címet a bejelentkezéshez
✓ Új regisztrációnál először erősítsd meg az email címedet!

5.3. ábra. Bejelentkezési felület

FIÓK

Regisztráció

Vezetéknév

Keresztnév

Email cím

Jelszó

Jelszó megerősítése

Lakcím (opcionális)

Regisztráció

Van már fiókod? [Bejelentkezés](#)

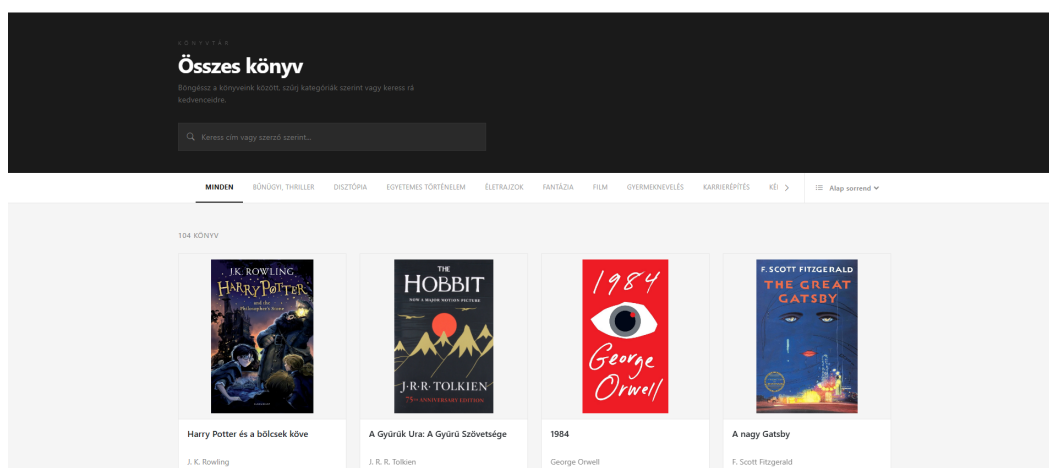
5.4. ábra. Regisztrációs felület

5.1.5. Könyvkatalógus (BooksPage)

A könyvkatalógus oldal az alkalmazás központi felülete, ahol a felhasználók böngészhetnek és szűrhetik az elérhető könyveket. A keresési és szűrési funkciók:

- Keresés cím és szerző alapján
- Kategóriaszűrő a könyvkatalógusban (20 kategória)
- Árszűrés és rendezés
- Valós idejű szűrés az eredmények azonnali frissítésével
- A backend `/api/books/by-price`, `/api/books/by-category/{kategoria}` és `/api/books/categories` végpontokon keresztül biztosítja a szűrt adatokat

Az egyes könyvek kártyákon (BookCard) jelennek meg, amelyek tartalmazzák a borítóképet, a címet, a szerzőt, az árat és a kategóriát. A részletes nézetben (BookDetails) a felhasználó megtekintheti a könyv teljes leírását, és hozzáadhatja a kosarához.



5.5. ábra. Könyvkatalógus oldal (BooksPage)

5.1.6. Kosár és fizetési rendszer (CartPage)

A projekt egyik legjelentősebb bővítése a teljes körű kosár- és fizetési rendszer megvalósítása.

Kosár funkciók:

- Könyvek hozzáadása a kosárhoz vásárlásra vagy bérlésre
- Kosár tételek mennyiségének módosítása
- Tételek eltávolítása és kosár ürítése

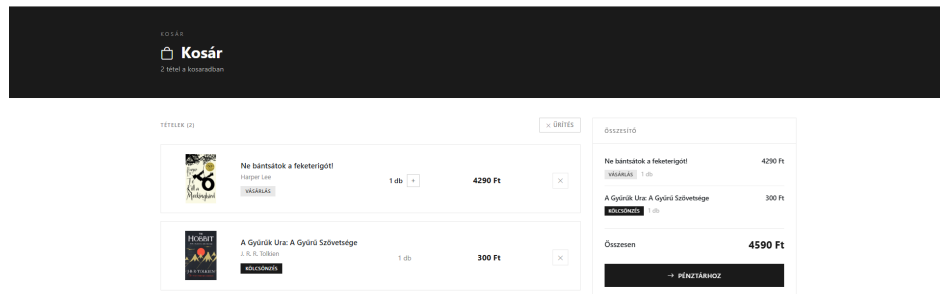
- Kosár összesítő automatikus számítása
- A kosár állapota a CartContext-ben kerül kezelésre, és a szerveren is perzisztálódik

Fizetési folyamat (Checkout):

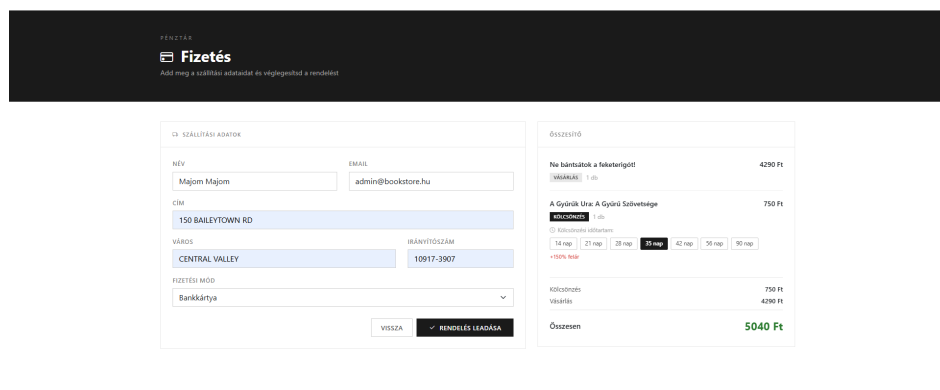
- A fizetési oldalon a felhasználó áttekintheti a rendelés tartalmát
- Rendelés típusok: bérlés, vásárlás, vagy vegyes
- Bérlés esetén a könyv árának 5%-a kerül felszámításra
- Vásárlás esetén a teljes ár kerül felszámításra, többszörös példány vásárlása lehetséges
- A checkout folyamat automatikusan létrehozza a fizetési rekordot, a bérlési rekordokat, és frissíti a készletállapotot

Fizetési nyilvántartás:

- A /api/payments végpont biztosítja a fizetési előzmények lekérdezését
- Napi bevétel összesítés a /api/payments/today-revenue végponton
- Felhasználónkénti fizetési előzmények



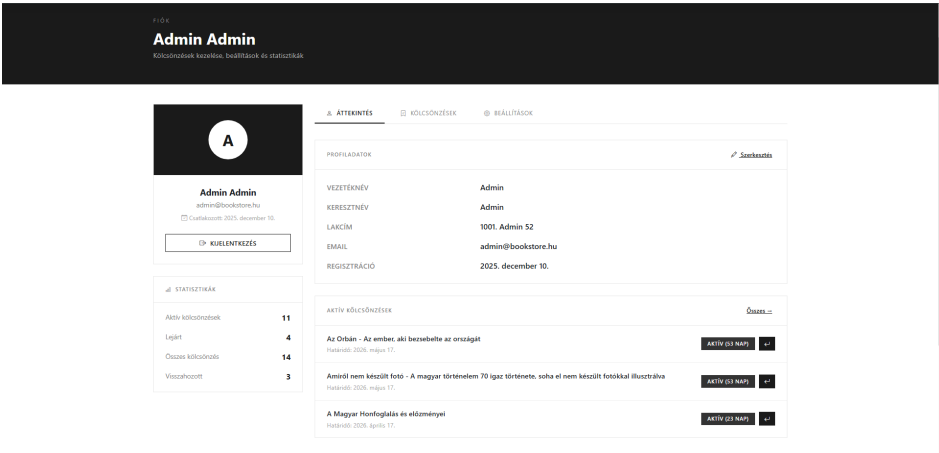
5.6. ábra. Kosár oldal (CartPage)



5.7. ábra. Fizetési felület (Checkout)

5.1.7. Profiloldal (ProfilePage)

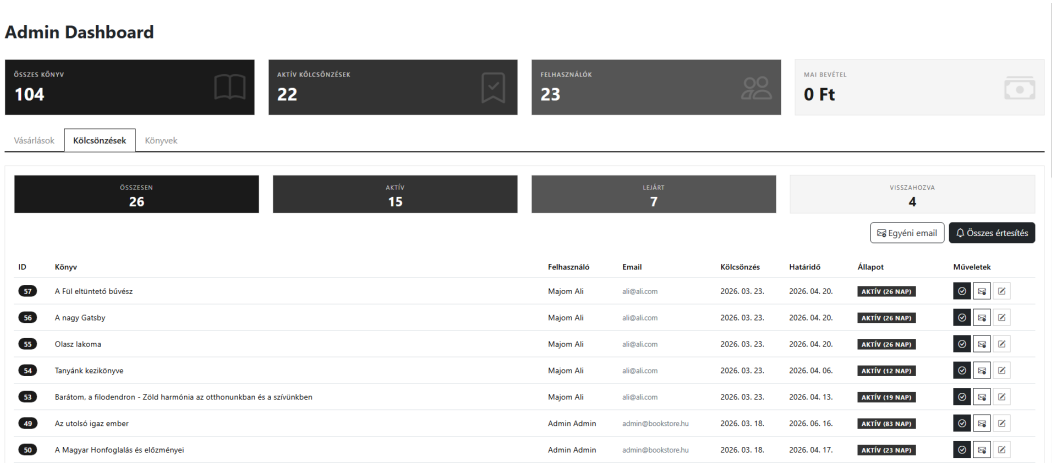
A felhasználói profil oldal lehetővé teszi a személyes adatok módosítását és a rendelési történet megtekintését. A profil oldalon a felhasználók módosíthatják a nevüket, e-mail címüket, címüket, és jelszavukat is megváltoztathatják. A profil kezelés során figyelembe vettük az adatvédelmi követelményeket és a felhasználói élményt.



5.8. ábra. Profiloldal (ProfilePage)

5.1.8. Adminisztrátori felület (AdminPage)

Az admin panel a weboldal kezelői számára készült felület, amely lehetővé teszi a könyvkatalógus, a felhasználók, a bérletek és a fizetések kezelését. Az admin felületen elérhető a Dashboard, amely összesítő statisztikákat jelenít meg, valamint könyv-, bérlet- és fizetéskezelő szekciókat tartalmaz.



5.9. ábra. Admin Dashboard

5.1.9. WPF karbantartó alkalmazás

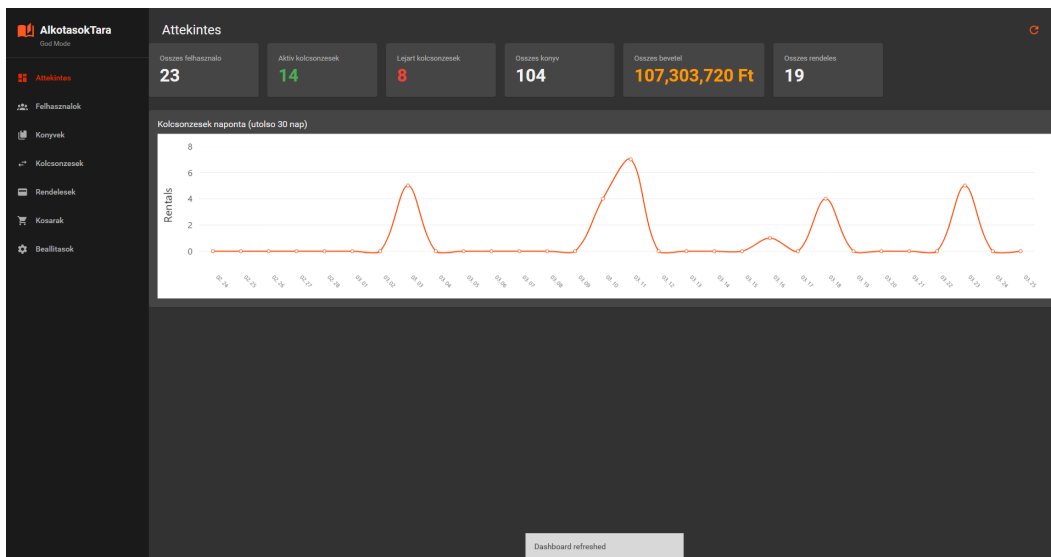
A WPF alapú karbantartó alkalmazás az adminisztrációs feladatok ellátására szolgál. Az alkalmazás neve „AlkotásokTára God Mode”, és a következő technológiai megoldásokra épül:

Technológiai frissítések:

- .NET 10.0 keretrendszerre történő migráció
- Material Design 3 megjelenés sötét témával és Deep Orange színsémával
- MVVM architektúra a CommunityToolkit.Mvvm segítségével
- Dependency Injection a szolgáltatások kezeléséhez

Az alkalmazás **7 fő szekciót** tartalmaz:

1. **Dashboard** – Összesítő nézet statisztikai kártyákkal: Összes felhasználó, aktív bérletek, lejárt bérletek, összes könyv, teljes bevétel, összes rendelés, 30 napos bérleti trend diagram (LiveCharts2)
2. **Felhasználók (Users)** – Felhasználókezelés: keresés név és e-mail alapján, részletes felhasználói nézet bérletekkel, kosár tartalommal és fizetési előzményekkel, felhasználó törlése
3. **Könyvek (Books)** – Könyvkatalógus kezelése: keresés cím és szerző alapján, szűrés kategória szerint, új könyv hozzáadása dialógusablakkal, könyvpéldányok létrehozása és törlése (automatikus leltári számgenerálás), meglévő könyvek szerkesztése
4. **Bérletek (Rentals)** – Bérleškövetés: státuszszűrők (Összes, Aktív, Lejárt, Visszahozott), keresés felhasználó neve, e-mailje vagy könyvcím alapján, összesítő statisztikák, bérlet visszahozottnak jelölése, emlékeztető értesítés küldése felhasználóknak, színkódolt státuszjelzők
5. **Rendelések/Fizetések (Orders)** – Fizetési előzmények: keresés felhasználó e-mail/név alapján, dátumtartomány szűrés, CSV exportálás funkcionalitás
6. **Kosarak (Carts)** – Bevásárlókosarak kezelése: összes felhasználói kosár megtekintése, kosár tartalmának és tételeinek megtekintése, kosár ürítése
7. **Beállítások (Settings)** – Konfigurációs panel: API Base URL beállítása, admin felhasználó ID beállítása, téma váltás, kapcsolat tesztelése, alkalmazás verzió megjelenítése



5.10. ábra. WPF karbantartó alkalmazás (God Mode Dashboard)

5.1.10. Bérleti rendszer és értesítések

A bérleti rendszer lehetővé teszi a könyvek ideiglenes kölcsönzését, alapértelmezetten 14 napos kölcsönzési időtartammal.

Bérleti funkciók:

- Kölcsönzés indítása a kosáron és checkout-on keresztül
- Bérlet egyenként 1 példányra korlátozott
- Bérleti állapotok: aktív, lejárt, visszahozott
- Visszahozás rögzítése az admin felületen

Automatikus e-mail értesítések:

- A backend egy háttérszolgáltatást (RentalNotificationService) futtat, amely ASP.NET Core Hosted Service-ként működik
- Emlékeztető e-mailek a közelgő lejáratú bérletekről
- Késedelmi értesítések a lejárt és vissza nem hozott könyvekről
- Az e-mailek a MailKit könyvtáron és SMTP protokollon keresztül kerülnek ki-küldésre

5.1.11. Példánykészlet-kezelés

A könyvek fizikai példányainak nyilvántartása a copies (példány) entitáson keresztül valósul meg:

- Minden példány egyedi leltári számmal rendelkezik (pl. HP1-0001, BOOK-00007-001, BK8-1771930120941-0002)
- A példányok elérhetősége (kölcönözhető/nem elérhető) külön nyomon követhető
- A vásárlás és bérlet során a rendszer ellenőrzi a készletállapotot
- A példányok kezelése az admin felületen és a WPF alkalmazásban is elérhető

5.1.12. Tesztelés és minőségbiztosítás

A frontend tesztelési keretrendszere a következő könyvtárakat tartalmazza:

- **@testing-library/react 16.3.0** – React komponensek tesztelése
- **@testing-library/jest-dom 6.9.1** – DOM állapot ellenőrzése
- **@testing-library/user-event 13.5.0** – Felhasználói interakciók szimulálása
- **web-vitals 2.1.4** – Teljesítmény monitorozás

5.2. Adatbázis

5.2.1. Bevezető

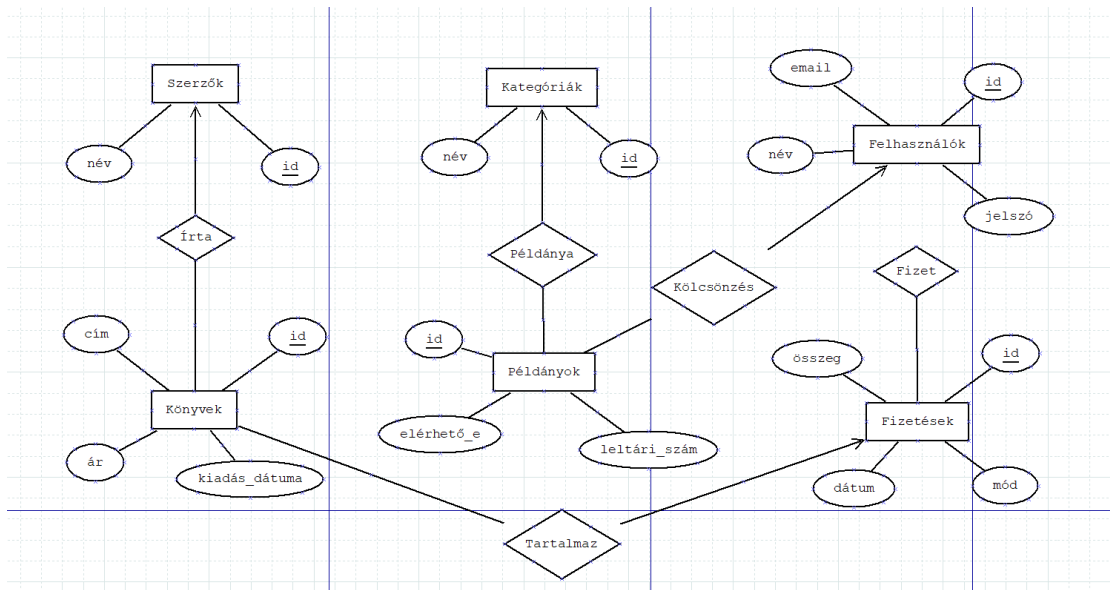
A projekt egyetlen, központi adatbázist használ, amely a teljes alkalmazás működéséhez szükséges adatokat tárolja. Az adatbázis MySQL (MariaDB 10.11) szerveren fut, utf8mb4 karakterkészlettel és utf8mb4_hungarian_ci rendezéssel a magyar nyelvű tartalom helyes kezeléséhez.

Az adatbázis tervezése során figyelembe vettük a normalizálási elveket. A fejlesztés során egy külön normalizálási migrációt hajtottunk végre, amelynek részeként:

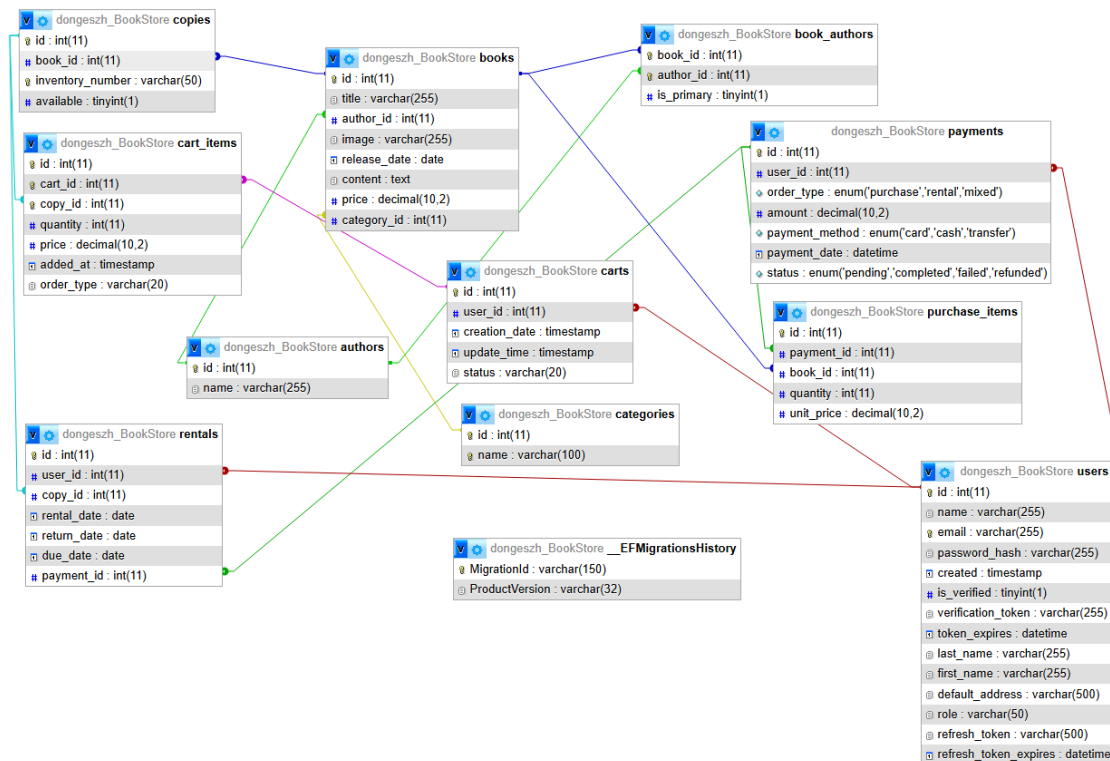
- A books.category varchar mezőt kiszerveztük egy külön categories táblába (3NF)
- Létrehoztuk a book_authors kapcsolótáblát a több szerzős könyvek kezelésére (1NF, many-to-many)
- A payments.order_type ENUM mezőt bővítettük a 'mixed' értékkel
- Teljesítmény indexeket adtunk hozzá a gyakran szűrt mezőkhöz
- A users.name mező szinkronizálására trigger hoztunk létre

5.2.2. Táblák részletes leírása

Az adatbázis jelenleg **11 fő táblát** tartalmaz:



5.11. ábra. ER diagram – az adatbázis entitásainak kapcsolata



5.12. ábra. Adatbázis séma diagram

authors tábla

A szerzők alapadatait tartalmazza.

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Szerző egyedi azonosítója
name	VARCHAR(255), NOT NULL	Szerző neve

5.2. táblázat. authors tábla

Az adatbázis jelenleg 76 szerzőt tartalmaz, például J.K. Rowling, J.R.R. Tolkien, George Orwell és számos magyar szerző.

books tábla

A könyvkatalógus központi táblája, amely az összes könyv alapadatait tartalmazza.

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Könyv egyedi azonosítója
title	VARCHAR(255), NOT NULL	Könyv címe
author_id	INT(11), FK → authors(id)	Elsődleges szerző azonosítója
image	VARCHAR(255)	Borítókép URL-je
release_date	DATE	Kiadási dátum
content	TEXT	Könyv tartalmi leírása
price	DECIMAL(10,2), NOT NULL	Könyv ára (Ft)
category_id	INT(11), FK → categories(id)	Kategória azonosítója

5.3. táblázat. books tábla

Indexek: author_id, category_id (FK index), title (keresési index). A tábla jelenleg 115 könyvet tartalmaz, 20 különböző kategóriában.

categories tábla

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Kategória egyedi azonosítója
name	VARCHAR(100), NOT NULL, UNIQUE	Kategória neve

5.4. táblázat. categories tábla

Mező	Típus	Leírás
book_id	INT(11), PK, FK → books(id)	Könyv azonosítója
author_id	INT(11), PK, FK → authors(id)	Szerző azonosítója
is_primary	TINYINT(1), DEFAULT 1	Elsődleges szerző jelzése

5.5. táblázat. book_authors tábla

Mező	Típus	Leírás
id	INT(11), PK, AU-TO_INCREMENT	Példány egyedi azonosítója
book_id	INT(11), FK → books(id)	A könyv azonosítója
inventory_number	VARCHAR(50), UNIQUE	Egyedi leltári szám
available	TINYINT(1), DEFAULT 1	Elérhetőségi állapot

5.6. táblázat. copies tábla

book_authors tábla (kapcsolótábla)

copies tábla

users tábla

Mező	Típus	Leírás
id	INT(11), PK, AU-TO_INCREMENT	Felhasználó azonosítója
name	VARCHAR(255), NOT NULL	Teljes név
email	VARCHAR(255), UNIQUE	E-mail cím
password_hash	VARCHAR(255)	BCrypt hash-elt jelszó
created	TIMESTAMP	Fiók létrehozása
role	VARCHAR(20), DEFAULT 'user'	Szerepkör (user/admin)
is_verified	TINYINT(1), DEFAULT 0	E-mail megerősítve
verification_token	VARCHAR(255)	Megerősítő token
token_expires	DATETIME	Token lejárat ideje
refresh_token	VARCHAR(255)	JWT refresh token
refresh_token_expires	DATETIME	Refresh token lejárat
last_name	VARCHAR(255)	Vezetéknév
first_name	VARCHAR(255)	Keresztnév
default_address	VARCHAR(500)	Szállítási cím

5.7. táblázat. users tábla

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Kosár azonosítója
user_id	INT(11), FK → users(id)	Felhasználó azonosítója
creation_date	TIMESTAMP	Létrehozás időpontja
update_time	TIMESTAMP	Utolsó módosítás
status	VARCHAR(20), DEFAULT 'active'	Kosár státusza

5.8. táblázat. carts tábla

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Tétel azonosítója
cart_id	INT(11), FK → carts(id)	Kosár azonosítója
copy_id	INT(11), FK → copies(id)	Példány azonosítója
quantity	INT(11), DEFAULT 1	Mennyiség
price	DECIMAL(10,2)	Ár
added_at	TIMESTAMP	Hozzáadás időpontja
order_type	VARCHAR(20), DEFAULT 'rental'	Rendelés típusa

5.9. táblázat. cart_items tábla

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Bérlés azonosítója
user_id	INT(11), FK → users(id)	Felhasználó azonosítója
copy_id	INT(11), FK → copies(id)	Példány azonosítója
rental_date	DATE, NOT NULL	Kölcsönzés kezdete
return_date	DATE	Visszahozás dátuma
due_date	DATE	Lejárat dátum
payment_id	INT(11), FK → payments(id)	Fizetés azonosítója

5.10. táblázat. rentals tábla

carts tábla

cart_items tábla

rentals tábla

payments tábla

purchase_items tábla

5.2.3. Táblakapcsolatok összefoglalása

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Fizetés azonosítója
user_id	INT(11), FK → users(id)	Felhasználó azonosítója
order_type	ENUM('purchase','rental','mixed')	Rendelés típusa
amount	DECIMAL(10,2)	Fizetett összeg
payment_method	ENUM('card','cash','transfer')	Fizetési mód
payment_date	DATETIME	Fizetés dátuma
status	ENUM('pending','completed','failed','refunded')	Státusz

5.11. táblázat. payments tábla

Mező	Típus	Leírás
id	INT(11), PK, AU- TO_INCREMENT	Tétel azonosítója
payment_id	INT(11), FK → payments(id)	Fizetés azonosítója
book_id	INT(11), FK → books(id)	Könyv azonosítója
quantity	INT(11), DEFAULT 1	Mennyiség
unit_price	DECIMAL(10,2)	Egységár

5.12. táblázat. purchase_items tábla

Kapcsolat	Típus	Leírás
authors ↔ books	1:N	Egy szerzőhöz több könyv
books ↔ authors	N:M	Több szerzős könyvek
categories → books	1:N	Kategória – könyvek
books → copies	1:N	Könyv – példányok
users → carts	1:N	Felhasználó – kosarak
carts → cart_items	1:N	Kosár – tételek
users → rentals	1:N	Felhasználó – kölcsönzések
users → payments	1:N	Felhasználó – fizetések
payments → purchase_items	1:N	Fizetés – tételek
payments → rentals	1:N	Fizetés – kölcsönzések

5.13. táblázat. Táblakapcsolatok

5.3. Backend

5.3.1. Bevezető

A BookStore backend egy ASP.NET Core 8 alapú REST API, amely a teljes üzleti logikát és adatkezelést biztosítja. A backend 8 fő kontrollert tartalmaz, amelyek a könyvek, szerzők, példányok, kosár, kölcsönzések, fizetések, felhasználók és hitelesítés kezelését végzik.

5.3.2. Hitelesítés és jogosultságkezelés (JWT)

A rendszer JSON Web Token (JWT) alapú hitelesítést alkalmaz, amely iparági szabvány a stateless autentikációhoz RESTful API környezetben. A hitelesítési folyamat két token típusra épül:

- **Access Token** – rövid élettartamú JWT (60 perc), amely minden védett API kéréshez szükséges
- **Refresh Token** – hosszú élettartamú token (7 nap), amellyel az access token megújítható bejelentkezés nélkül

JWT token szerkezete és generálás

Az access token generálása a `JwtService` szolgáltatásban történik. A token a következő claim-eket (állításokat) tartalmazza:

Claim	Leírás
UserId	A felhasználó egyedi azonosítója
Email	A felhasználó e-mail címe
Name	A felhasználó megjelenített neve
Role	A felhasználó szerepe (user / admin)
NameIdentifier	Másodlagos azonosító

5.14. táblázat. JWT token claim-ek

A token aláírása **HMAC-SHA256** algoritmussal történik, szimmetrikus titkos kulcs használatával. A konfigurációs paraméterek az `appsettings.json` fájlban kerülnek meghatározásra:

Listing 5.1. JWT konfiguráció (appsettings.json)

```
"Jwt": {  
  "Secret": "BookStore_SuperSecret_JWT_Key_...",  
  "Issuer": "BookStoreAPI",  
  "Audience": "BookStoreFrontend",  
  "AccessTokenExpirationMinutes": 60  
}
```

Token validálás

A backend minden védett végpontnál ellenőrzi a bejövő JWT tokenet. A validálási paraméterek:

- **ValidateIssuer**: true – az Issuer mezőnek meg kell egyeznie („BookStoreAPI”)
- **ValidateAudience**: true – az Audience mezőnek meg kell egyeznie („BookStoreFrontend”)
- **ValidateLifetime**: true – lejárt token elutasítása
- **ValidateIssuerSigningKey**: true – az aláírás ellenőrzése
- **ClockSkew**: TimeSpan.Zero – nincs időeltérés-tolerancia

Regisztrációs folyamat

1. A felhasználó megadja adatait (név, vezetéknév, keresztnév, e-mail, jelszó, cím)
2. A jelszó **BCrypt** algoritmussal hash-elődik a szerveren
3. A rendszer generál egy 24 órás érvényességű verifikációs tokenet
4. E-mail megerősítő link küldése a MailKit könyvtáron keresztül
5. A fiók aktiválás előtt a bejelentkezés nem lehetséges

Bejelentkezési folyamat

1. A felhasználó elküldi az e-mail/jelszó kombinációt (**POST /api/auth/login**)
2. A backend ellenőrzi a BCrypt hash-t és az e-mail verifikáció státuszát
3. Sikeres hitelesítés esetén generálódik egy JWT access token és egy refresh token
4. A refresh token az adatbázisban tárolódik (users tábla: **refresh_token**, **refresh_token_expires**)
5. A válasz tartalmazza mindkét tokenet és a felhasználó adatait

Token megújítás (Refresh Token flow)

Amikor az access token lejár, a kliens automatikusan megújítja azt:

1. A frontend észleli a 401 Unauthorized választ
2. Elküldi a lejárt access tokenet és a refresh tokenet a **PATCH /api/auth/refresh-token** végpontra

3. A backend validálja a refresh token (érvényesség, adatbázis egyezés)
4. A lejárt access tokenből kinyeri a felhasználó adatait (aláírás-ellenőrzéssel, lejárat nélkül)
5. Új access token és refresh token generálódik
6. Az eredeti API kérés megismétlődik az új tokennel

A frontend egy **várakozási sor** (subscriber queue) mintát alkalmaz a párhuzamos kérések kezelésére: ha több API hívás egyszerre kap 401-es választ, csak egyetlen token megújítás történik, és a többi kérés megvárja az új tokenet.

Token tárolás a frontenden

A webes felület a tokeneket a böngésző `localStorage`-ában tárolja:

- `localStorage.accessToken` – JWT access token
- `localStorage.refreshToken` – refresh token
- `localStorage.currentUser` – a felhasználó adatai (JSON)

Minden autentikált API kérés az `authFetch()` wrapper függvényen keresztül történik, amely automatikusan hozzáfűzi az `Authorization: Bearer {accessToken}` fejléceket a HTTP kérésekhez.

Token kezelés a Desktop alkalmazásban

A WPF alkalmazás az `AuthService` szolgáltatáson keresztül kezeli a tokeneket:

- A tokenek memóriában tárolódnak (nem fájlrendszerben)
- A `HttpClient Authorization` fejléce automatikusan frissül bejelentkezéskor
- Az `ApiService` automatikus token megújítást végez 401-es válasz esetén (`SendWithRetryAsync`)
- Csak **admin** szerepű felhasználók léphetnek be a desktop alkalmazásba

Kijelentkezés

1. A kliens elküldi a `POST /api/auth/logout` kérést
2. A backend törli a refresh tokenet az adatbázisból (érvénytelenítés)
3. A frontend törli a `localStorage` tartalmát (access token, refresh token, felhasználó)
4. Az `AuthContext` frissül és a felhasználó kijelentkezettetésre kerül

5.3.3. Könyvkezelés

A könyvek kezelése a BooksController-en keresztül történik, amely CRUD műveleteket biztosít.

5.3.4. Szerzők kezelése

Az AuthorController a szerzők listázását és új szerzők létrehozását biztosítja.

5.3.5. Példánykezelés

A CopiesController a könyvpéldányok kezelését végzi: összes példány és könyv szerinti lekérdezés, új példány létrehozása és példány törlése.

5.3.6. Kosár és fizetés

A CartController kezeli a kosár műveleteket, míg a PaymentsController a fizetési tranzakciókat.

5.3.7. Kölcsönzések kezelése

A RentalsController biztosítja a bérletek kezelését. A backend egy háttérszolgáltatást (RentalNotificationService) futtat ASP.NET Core Hosted Service-ként.

5.3.8. Felhasználók kezelése

A UsersController a felhasználók listázását, profil módosítását és törlését biztosítja.

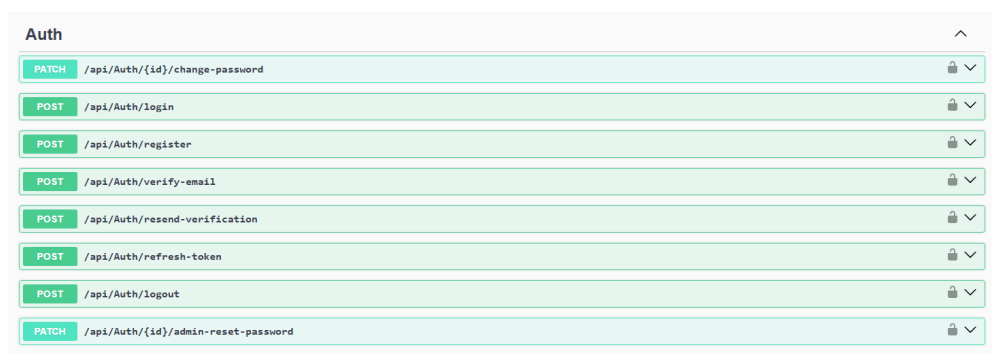
6. fejezet

REST API végpontok összefoglalása

6.1. AuthController (/api/auth)

Metódus	Végpont	Funkció
POST	/api/auth/register	Új felhasználó regisztrálása
POST	/api/auth/login	Bejelentkezés
POST	/api/auth/verify-email	E-mail megerősítése
POST	/api/auth/resent-verification	Megerősítő e-mail újraküldése
PATCH	/api/auth/refresh-token	Access token megújítása
PATCH	/api/auth/{id}/change-password	Jelszó módosítása

6.1. táblázat. AuthController végpontok



6.1. ábra. AuthController – Swagger nézet

6.2. BooksController (/api/books)

Metódus	Végpont	Funkció
GET	/api/books	Összes könyv lekérdezése
GET	/api/books/{id}	Könyv lekérdezése ID alapján
POST	/api/books	Új könyv létrehozása
PUT	/api/books/{id}	Könyv módosítása
DELETE	/api/books/{id}	Könyv törlése
GET	/api/books/by-price	Ár szerinti szűrés
GET	/api/books/by-category/{kategoria}	Kategória szerinti szűrés
GET	/api/books/categories	Kategóriák listázása
GET	/api/books/price-stats	Ár statisztikák

6.2. táblázat. BooksController végpontok

The image shows a Swagger UI interface for the BooksController. It lists the following endpoints with their methods and descriptions:

- GET /api/Books
- POST /api/Books
- GET /api/Books/by-price
- GET /api/Books/by-category/{kategoria}
- GET /api/Books/categories
- GET /api/Books/price-stats
- GET /api/Books/{id}
- PUT /api/Books/{id}
- DELETE /api/Books/{id}

6.2. ábra. BooksController – Swagger nézet

6.3. AuthorController (/api/author)

Metódus	Végpont	Funkció
GET	/api/author	Szerzők listázása
GET	/api/author/{id}	Szerző lekérdezése ID alapján
POST	/api/author	Új szerző létrehozása
PUT	/api/author/{id}	Szerző módosítása
DELETE	/api/author/{id}	Szerző törlése

6.3. táblázat. AuthorController végpontok

The image shows a Swagger UI interface for the AuthorController. It lists the following endpoints with their methods and descriptions:

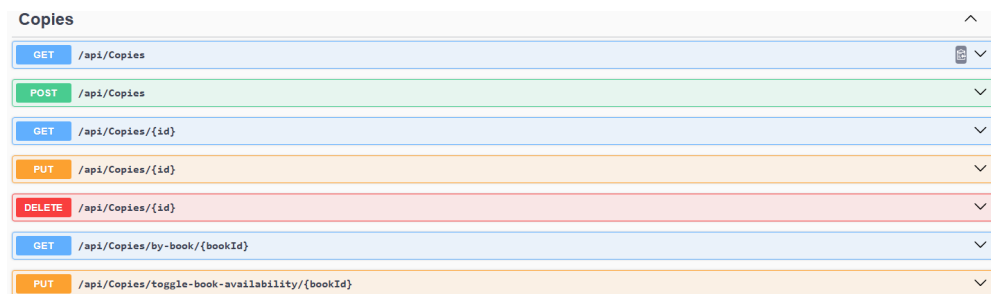
- GET /api/Author
- POST /api/Author
- GET /api/Author/{id}
- PUT /api/Author/{id}
- DELETE /api/Author/{id}

6.3. ábra. AuthorController – Swagger nézet

6.4. CopiesController (/api/copies)

Metódus	Végpont	Funkció
GET	/api/copies	Összes példány
GET	/api/copies/{id}	Példány lekérdezése ID alapján
GET	/api/copies/by-book/{bookId}	Könyv példányai
POST	/api/copies	Új példány létrehozása
PUT	/api/copies/{id}	Példány módosítása
DELETE	/api/copies/{id}	Példány törlése
PUT	/api/copies/toggle-book-availability/{bookId}	Elérhetőség váltása

6.4. táblázat. CopiesController végpontok



6.4. ábra. CopiesController – Swagger nézet

6.5. CartController (/api/cart)

Metódus	Végpont	Funkció
GET	/api/cart/my-cart	Saját kosár lekérdezése
GET	/api/cart/{id}	Kosár lekérdezése ID alapján
GET	/api/cart	Összes kosár (admin)
POST	/api/cart/add	Tétel hozzáadása
DELETE	/api/cart/item/{id}	Tétel eltávolítása
DELETE	/api/cart/clear	Kosár ürítése
POST	/api/cart/checkout	Rendelés véglegesítése

6.5. táblázat. CartController végpontok

Cart	
GET	/api/Cart/my-cart
GET	/api/Cart/{id}
POST	/api/Cart/add
DELETE	/api/Cart/item/{cartItemId}
DELETE	/api/Cart/clear
GET	/api/Cart
POST	/api/Cart/checkout

6.5. ábra. CartController – Swagger nézet

6.6. RentalsController (/api/rentals)

Metódus	Végpont	Funkció
GET	/api/rentals	Összes bérlet
GET	/api/rentals/{id}	Bérlet ID alapján
GET	/api/rentals/user/{userId}	Felhasználó bérletei
PATCH	/api/rentals/{id}/return	Visszahozás jelölése
POST	/api/rentals/send-notifications	Értesítések küldése
POST	/api/rentals/{id}/send-reminder	Emlékeztető küldése
POST	/api/rentals/send-custom-email	Egyéni e-mail küldése
POST	/api/rentals	Új bérlet létrehozása
PUT	/api/rentals/{id}	Bérlet módosítása
DELETE	/api/rentals/{id}	Bérlet törlése

6.6. táblázat. RentalsController végpontok

Rentals	
GET	/api/Rentals
POST	/api/Rentals
GET	/api/Rentals/{id}
PUT	/api/Rentals/{id}
DELETE	/api/Rentals/{id}
GET	/api/Rentals/user/{userId}
PATCH	/api/Rentals/{id}/return
POST	/api/Rentals/send-notifications
POST	/api/Rentals/{id}/send-reminder
POST	/api/Rentals/send-custom-email

6.6. ábra. RentalsController – Swagger nézet

6.7. PaymentsController (/api/payments)

Metódus	Végpont	Funkció
GET	/api/payments	Összes fizetés
GET	/api/payments/{id}	Fizetés ID alapján
GET	/api/payments/user/{userId}	Felhasználó fizetései
POST	/api/payments	Új fizetés
PUT	/api/payments/{id}/status	Státusz módosítása
GET	/api/payments/status/{status}	Fizetések státusz alapján
GET	/api/payments/today-revenue	Mai bevétel
GET	/api/payments/purchases	Vásárlások listázása

6.7. táblázat. PaymentsController végpontok

The screenshot shows the Swagger UI for the PaymentsController. It lists the following endpoints with their methods and descriptions:

- GET /api/Payments: Összes fizetés
- POST /api/Payments: Új fizetés
- GET /api/Payments/{id}: Fizetés ID alapján
- DELETE /api/Payments/{id}: Fizetés törlése
- GET /api/Payments/user/{userId}: Felhasználó fizetései
- GET /api/Payments/status/{status}: Fizetések státusz alapján
- PUT /api/Payments/{id}/status: Státusz módosítása
- GET /api/Payments/today-revenue: Mai bevétel
- GET /api/Payments/purchases: Vásárlások listázása

6.7. ábra. PaymentsController – Swagger nézet

6.8. UsersController (/api/users)

Metódus	Végpont	Funkció
GET	/api/users	Felhasználók listázása (admin)
GET	/api/users/{id}	Felhasználó lekérdezése
POST	/api/users	Új felhasználó (admin)
PUT	/api/users/{id}	Felhasználó módosítása
PATCH	/api/users/{id}/role	Szerepkör módosítása (admin)
PATCH	/api/users/{id}/profile	Profil módosítása
DELETE	/api/users/{id}	Felhasználó törlése

6.8. táblázat. UsersController végpontok

The screenshot shows the Swagger UI for the UsersController. It lists the following endpoints with their methods and descriptions:

- GET /api/Users: Felhasználók listázása (admin)
- POST /api/Users: Új felhasználó (admin)
- GET /api/Users/{id}: Felhasználó lekérdezése
- PUT /api/Users/{id}: Felhasználó módosítása
- DELETE /api/Users/{id}: Felhasználó törlése
- PATCH /api/Users/{id}/role: Szerepkör módosítása (admin)
- PATCH /api/Users/{id}/profile: Profil módosítása

6.8. ábra. UsersController – Swagger nézet

7. fejezet

Projektstruktúra

```
Prodzsekt/
|-- BookStore-Frontend/                                (React frontend)
|   +-- src/
|       |-- api.jsx, App.jsx, index.jsx
|       |-- components/
|           |-- admin/ (Dashboard, BookManagement, ...)
|           |-- auth/ (Login, Register, EmailVerification)
|           |-- books/ (BookCard, BookDetails, BookList, ...)
|           |-- cart/ (Cart, CartItem, Checkout)
|           |-- common/ (Navbar, Footer, SearchBar, ...)
|           +-- user/ (Profile)
|       |-- context/ (AuthContext, CartContext, ToastContext)
|       |-- hooks/ (useScrollAnimation)
|       +-- pages/ (HomePage, BooksPage, CartPage, ...)
|
|-- BookStore-Backend/                                (ASP.NET Core backend)
|   +-- Backend/
|       |-- Api/Controllers/ (8 controller)
|       |-- Application/DTOs/, Mappers/
|       |-- Domain/Model/ (entitasok, DbContext)
|       |-- Services/ (Email, Jwt, RentalNotification)
|       +-- Backend.Tests/
|
+-- BookStore-Desktop/                                (WPF alkalmazas)
    +-- God.Support.mode/
        |-- Converters/, Models/, Services/
        |-- ViewModels/, Views/
    +-- App.xaml, MainWindow.xaml
```

Irodalomjegyzék

- [1] React, „React — The library for web and native user interfaces,” [Online]. Available: <https://react.dev/>.
- [2] Microsoft, „ASP.NET Core Documentation,” [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/>.
- [3] Microsoft, „Entity Framework Core,” [Online]. Available: <https://learn.microsoft.com/en-us/ef/core/>.
- [4] Bootstrap, „Bootstrap 5 Documentation,” [Online]. Available: <https://getbootstrap.com/docs/5.3/>.
- [5] React Router, „React Router Documentation,” [Online]. Available: <https://reactrouter.com/>.
- [6] Oracle, „MySQL Documentation,” [Online]. Available: <https://dev.mysql.com/doc/>.
- [7] AutoMapper, „AutoMapper Documentation,” [Online]. Available: <https://automapper.org/>.
- [8] FluentValidation, „FluentValidation Documentation,” [Online]. Available: <https://docs.fluentvalidation.net/>.
- [9] J. Stedfast, „MailKit,” [Online]. Available: <https://github.com/jstedfast/MailKit>.
- [10] Serilog, „Serilog — Simple .NET logging,” [Online]. Available: <https://serilog.net/>.
- [11] BCrypt.Net, „BCrypt.Net,” [Online]. Available: <https://github.com/BcryptNet/bcrypt.net>.
- [12] Microsoft, „WPF Documentation,” [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>.
- [13] Material Design In XAML, „Material Design In XAML Toolkit,” [Online]. Available: <https://github.com/MaterialDesignInXAML/MaterialDesignInXamlToolkit>.

- [14] Microsoft, „CommunityToolkit.Mvvm,” [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/>.
- [15] LiveCharts, „LiveCharts2,” [Online]. Available: <https://livecharts.dev/>.
- [16] SmartBear, „Swagger,” [Online]. Available: <https://swagger.io/docs/>.
- [17] RFC 7519, „JSON Web Token (JWT),” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>.